

Paul D. Martin, Ph.D.

Chief Scientist, Harbor Experts · Lecturer in Computer Science, Johns Hopkins University

Professional Background

I am the Chief Scientist at Harbor Experts, Inc. and a part-time Lecturer in the Department of Computer Science at Johns Hopkins University.

I completed my B.S., M.S.E., and Ph.D. degrees at Johns Hopkins University, with all degrees, including my doctorate, being in computer science.

I also serve as Secretary of the IEEE 7024 Working Group, which is developing the Standard for the Procurement, Verification and Validation, and Life Cycle Management of Forensic Technologies. I hold Six Sigma Black Belt and Green Belt certifications, which include training in measurement, statistical analysis, root cause analysis, and quality management.

Teaching

My teaching at Johns Hopkins University has included computer and network security, applied cryptography, and C and C++ programming, including developer tools and Linux environments. In the past I have taught classes on hardware hacking, including analyzing, modifying, and repairing computer hardware and embedded devices, as well as vulnerability analysis and exploitation. Earlier in my academic career, I served as a course assistant for Practical Cryptographic Systems.

My classes cover a variety of topics including hardware and software security design, analysis and reverse engineering for vulnerability assessment; hardware- and software-based attacks on computer components including RAM and CPUs; applied cryptography; computer architecture; computer networking; and component-level repair and modification. My classes also use hardware and software emulation and virtualization to present students with a variety of environments in which to learn and perform hands-on projects. In my security courses, students read real codebases to identify potential vulnerabilities and then validate their findings by writing working exploits. In 2026, I developed a module on large language model security for my course.

I co-instructed a short course called “Introduction to Hardware Hacking” with Dr. Michael Rushanan. We offered lessons on modifying game consoles and device firmware, electronics repair, binary analysis and modification, network traffic analysis, and web-based vulnerability assessment and exploitation. I also received the Computer Science Department’s Outstanding Teaching Assistant Award in 2014.

Research

In spring 2011, I completed my bachelor’s in computer science at Johns Hopkins University. In fall 2011, I began working towards my Ph.D., also at Johns Hopkins, where I researched computer security and privacy systems. My research focused on designing technologies for securing computing systems and presenting those technologies in ways that non-technical users could understand and use.

Part of my research focused on generating virtualized computer networks to provide malware education environments for students. I also developed an emulator for analyzing malware code segments as part of a student project with the NSA. This emulator was integrated into an IDA Pro plugin and was able to observe how binary instruction snippets taken from malware samples modified their computing environment. I also built a clustered virtualization infrastructure for my laboratory at Johns Hopkins.

During my tenure as a Ph.D. student, I was fully funded as a research assistant and/or teaching assistant throughout all of my semesters. I was a member of the Upsilon Pi Epsilon computer science honor society and its treasurer for the 2014-2015 school year, as well as a member of the Johns Hopkins chapter of the Association for Computing Machinery. I represented the department at open houses and admitted student days, spoke to its external advisory board, and served as faculty liaison czar, attending faculty meetings and reporting back to students. I also supervised undergraduate research assistants and mentored a high school student on a year-long software design project.

I finished my Ph.D. by successfully defending my dissertation, entitled “Securing Medical Devices and Protecting Patient Privacy in the Technological Age of Healthcare,” on February 12, 2016. My doctoral thesis describes a healthcare practice in which security technology is straightforward for healthcare providers to use. It focuses on the relationship between usability and security, with an emphasis on simplicity, automation, and error-proofing of security controls.

During my doctorate and afterward, I have conducted and published research in fingerprinting, anomaly detection, multifactor authentication, and embedded systems security. Some of this research has formed the basis for commercial products and services. For example, in my work on integrated audit and access control, which was funded in part by Accenture Labs, I developed a Hadoop-based application to perform large-scale statistical analysis of audit logs from an electronic medical record system. I also designed a web application to automatically produce human-readable reports and graphs for use in consulting. The work addressed using audit information to support minimum-necessary access to patient information and led to two patents assigned to Accenture.

Similarly, in my research at Applied Communication Sciences in 2013, I designed and implemented a web-based traffic visualization dashboard and analysis system for field area smart grid networks. It could be used to understand the current state of a network and detect unexpected anomalies. The dashboard aggregated and visualized multiple data streams in real time. Applied Communication Sciences subsequently patented this work and continued to build on the project, incorporating it into its SecureSmart Managed Security Service offering.

I have also worked on and published two research projects related to improving the security and ease of use of authentication technologies, particularly in healthcare settings. In one project, my co-authors and I designed an indoor location tracking system using Bluetooth Low Energy beacons designed to prevent spoofing. These beacons were placed around a building and mapped to a backend so that a user’s location could be used as a secondary authentication mechanism for accessing patient medical records. In the system we designed, a doctor would log into a mobile device connected to the backend and be presented with records of nearby patients as the doctor walked past their rooms.

In the other project, my co-authors and I designed an authentication bracelet that received a Kerberos ticket upon login to a modified computer terminal through low-energy electrical signals transmitted over the wearer's skin. The ticket could then be transferred to other terminals through a custom contact, allowing the wearer to log in without entering a password again. The bracelet was designed to lose its cryptographic secret when removed from the wearer. This work was published at Financial Cryptography and Data Security. My other cryptography-related research includes a study of methods for classifying network protocol implementation versions using OpenSSL.

My research after my doctorate has included embedded systems security, with an emphasis on binary analysis, anomaly detection, and automated security enforcement. I designed Sentinel, a security system that can be soldered directly to an embedded device's CPU to build execution profiles and enforce control-flow integrity. This work was published at the ACM/IEEE International Conference on Internet-of-Things Design and Implementation. I have also researched automated vulnerability analysis of containers and virtual appliances and comparisons of firmware vulnerabilities across and within product classes.

My current research includes software supply-chain security and the relationship between a component's reported version and its vulnerability status. This includes examining patch differences and vulnerability exposure in rolling and enterprise Linux distributions. I also study the use of large language models for technical research and analysis, including vulnerability analysis, code understanding, and forensic reconstruction, and the use of multimodal models to investigate disease progression.

I designed and built a local inference system for large open-weight language models, including mixture-of-experts models with hundreds of billions of parameters. The system combines an AMD EPYC processor, two terabytes of system memory, and four GPUs with a total of 128 gigabytes of video memory. I use this system to investigate how model architecture, quantization, memory bandwidth, and the allocation of work between CPUs and GPUs affect inference performance. I also use it to evaluate models' ability to complete tasks through sequences of tool use and other actions.

In this work, I measure prompt-processing and token-generation performance separately, develop models of expected performance, and compare those predictions with experimental results. I have investigated processing-batch size, context length, placement of model components in memory, and speculative decoding. I also modified scheduling behavior in llama.cpp to distribute offloaded computations across multiple GPUs and remove an unnecessary data transfer during prompt processing, improving throughput in the tested configuration.

I maintain a public laboratory notebook documenting these experiments, including hardware configurations, software builds, model files, raw measurements, and hypotheses that testing did not support. I have identified benchmarking issues that changed effective test conditions and repeated measurements under common conditions to distinguish the effects of software updates, hardware changes, and optimization techniques. Running these experiments locally allows me to control the models, prompts, data, and system configurations used in the analysis.

In 2023 and 2024, I served as a member of the program committee for the IEEE Symposium on Security and Privacy. I peer reviewed research submitted for publication at the Symposium, including work on

cryptographic protocols, memory forensics, side-channel attacks, password vaults, differential privacy, computational theory, memory module design, and hardware security.

Patents

I am a named inventor on five U.S. patents. These concern healthcare privacy through integrated audit and access control, network traffic profiling and visualization, and extraction of information from binary files for use in database queries. A full listing of these patents can be found on my curriculum vitae.

Industry

In 2007, I developed a Linux distribution and associated LiveCD build system intended to support identification and removal of malware from Windows computers. The distribution was based on Gentoo and compiled from source for use on computers with limited resources. I tuned the kernel and operating system configuration to reduce memory requirements, runtime overhead, and binary size.

I began working in the software industry in 2008, when I obtained my first independent consulting client, Brandeis University. I created a program that connected its computer repair shop's ticketing system to a label printer so that staff could automatically print labels identifying customers' devices. Shortly afterward, I began working for the Johns Hopkins University Digital Research and Curation Center as a student programmer. I created a batch importer for the DSpace digital archiving system and ported a web interface for statistics tracking and visualization to the same system so that library staff could examine its usage data.

In 2009, I began working for Independent Security Evaluators in Baltimore, Maryland. I worked on security and privacy projects involving cryptographic implementations, reverse engineering, fuzz testing, and antivirus and antimalware software. In one authorized assessment, I analyzed the iOS and Android applications of a digital-magazine distribution system used by a major Asian telecommunications provider and found multiple independent ways to bypass its digital-rights-management protections. My other work included testing cryptographic secret-splitting implementations, developing browser extensions for a cryptographic proxy system, and writing and debugging C++ and Python tests for proprietary disk-encryption software. I also reverse engineered binary file formats and developed tools and plugins for fuzz testing.

At Independent Security Evaluators, I designed and built an automated system capable of testing antivirus products against thousands of malware samples for Consumer Reports' annual antivirus analysis. I managed the VMware ESXi server on which the system ran. The system automated testing in Windows virtual machines, including snapshot creation and restoration, so that tests could be repeated and the machines returned to a known state. I worked for Independent Security Evaluators during semesters and summers until August 2011.

In 2010, I designed a security architecture for a large-scale digital curation system intended as an inter-university successor to existing digital repositories. In 2011, I performed a penetration test and security assessment for the University of Michigan of a cloud-based research system intended to hold confidential research information. The system used Amazon EC2 and VMware infrastructure, and my

work included penetration testing of VMware ESXi. I developed a formal threat model addressing potential attack vectors and prepared reports documenting my testing.

In February 2016, I joined Harbor Labs full-time as a research scientist. In 2018, I was promoted to Director of Firmware Analysis, and in December 2023, to Vice President of Applied Research and Technology. In October 2024, the division of Harbor Labs that I managed became Harbor Experts, and I became its Chief Scientist. At Harbor Experts, I manage technical analysis engagements and lead research projects.

A substantial portion of my work involves directing and performing source code reviews. I have reviewed systems ranging from thousands to billions of lines of code, including security products, television set-top boxes, network appliances, web-based enterprise systems, email management systems, telephony products, embedded system bootloaders, social network platforms, and virtualization platforms. This work has involved examining software architecture, tracing implemented functionality, and reconstructing development history from code and supporting records.

I developed a methodology for evaluating whether software was copied or derived from another implementation, including whether one codebase was used as a reference in developing another. The methodology combines development records, manual source code analysis, automated comparison, and binary analysis where source code is unavailable. It examines such evidence as architecture, dependencies, program structure, embedded strings, shared errors, and development history, and includes manual investigation of apparent matches to identify false positives. I have applied this methodology in client engagements and documented it in a whitepaper on source code comparison.

I separately developed and documented a methodology for assessing source code quality. It addresses the reliability of an implementation, development and testing practices, duplicated and unreachable code, error and exception handling, and input validation. My assessments also consider the materials needed to build, run, and test the software, including dependencies, build instructions, version information, and records of defects and unexpected results. Where appropriate, I evaluate software and development practices against applicable regulatory requirements, technical standards, and recognized guidance, including the IEEE Computer Society's Guide to the Software Engineering Body of Knowledge (SWEBOK).

At Independent Security Evaluators and Harbor Labs, companies engaged me to assess the security of products before and after market release. A substantial part of this work involved cryptographic protections. I identified the cryptographic invariants and security policies a product was intended to preserve, examined their implementation, and designed tests to determine whether those properties could be violated. This work involved source code analysis, reverse engineering, and practical testing of the protections in the product.

My medical-device work includes writing production code deployed in FDA-regulated devices and designing cryptographic protocols implemented in production healthcare environments. I have also identified and demonstrated remote-code-execution vulnerabilities in medical systems. In an assessment of a commercially deployed infusion-pump system, I found multiple vulnerabilities and developed a working exploit that achieved arbitrary code execution on the device.

At Harbor Labs, I was the technical and development lead for Firmware IQ, a commercial platform for automated security analysis of firmware, container images, and virtual-machine appliances. In one configuration, a developer uploads an image through a web-based portal, which uses a broker to forward it to an analysis engine. The engine unpacks the image into its components and performs more than one hundred checks for vulnerabilities, security weaknesses, and indicators of compromise. Results are returned through JSON to a web-based presentation system. The platform supported examination of firmware updates before deployment and examination of firmware extracted from devices for evidence that they had been compromised.

Firmware IQ's checks included vulnerable software components, insecure configurations, hardcoded credentials, and cryptographic implementation problems such as weak key generation and improper use of initialization vectors. As part of the product, I developed a technique for identifying software components and their versions from binaries and cross-referencing them with a vulnerability database, including NIST's National Vulnerability Database. I wrote the patent specification for this technique and am the named inventor on the resulting patent.

I have extracted and reverse engineered firmware from dozens of devices as part of my commercial firmware-security work, including the development of Firmware IQ. I maintain a hardware analysis laboratory that I use for firmware extraction and reverse engineering, signal capture, protocol decoding, and component-level investigation and repair. It includes oscilloscopes, logic analyzers, microscopes, programmable power supplies, soldering stations, and rework equipment. My experience spans a range of CPU architectures, from small microcontrollers through embedded processors to desktop and server-class systems. My work also includes open-source firmware and Coreboot on laptop and embedded platforms.

My reverse-engineering work also includes desktop and laptop computers, mobile phones, and their software. I examine compiled implementations and system behavior when source code is unavailable and develop tools and experiments to investigate how software operates. I have also evaluated the source code of at least ten Model Context Protocol (MCP) servers, which provide interfaces between AI applications and external tools and services.

Since Firmware IQ, I have incorporated lessons from analyzing firmware and operating systems into my own Linux distribution, Technicomp Benchtop Linux, which I launched in 2026. It is designed to support technical work including software development, security and forensic analysis, cryptography, machine learning, data science, system administration, and virtualization. My work includes integrating and configuring the operating system and its software environment, with attention to stability, reliability, and performance.

As part of this work, I have performed latency testing and worked on the design of low-latency Linux desktops. I investigate responsiveness while background workloads compete for CPU time, memory, and storage bandwidth, including file transfers, video encoding, and parallel compilation. My work addresses kernel preemption, scheduling, interrupt handling, memory reclamation, compressed swap, storage scheduling, writeback behavior, and power management. I also consider low-latency audio and the requirements of applications that must meet timing deadlines.

My testing approach distinguishes throughput from responsiveness and considers the distribution of delays, including infrequent stalls. My Linux design notes set out baseline and loaded conditions, frame timing, input responsiveness, scheduling-latency percentiles, and worst-case observations, together with throughput comparisons to assess the effects of tuning changes. I document experimental conditions and results so that I can repeat comparisons and investigate the cause of a measured difference.

I also design and operate virtualization, storage, compute, and network infrastructure for my research and technical work. In addition to the Proxmox cluster I built at Johns Hopkins, I operate a Proxmox cluster at home for AI agent swarms. My environments include virtual machines, LXC containers, isolated test networks, and virtualized storage and network appliances. I use PCIe passthrough to give selected virtual machines direct access to physical hardware, and ZFS for storage integrity checking, snapshots, and replication. I build and operate OpenBSD and OPNsense network appliances for traffic inspection, traffic shaping, and segmentation. This infrastructure allows me to create controlled environments for malware and firmware analysis and to repeat experiments across operating systems and software configurations.